

# **Document Intelligence**

AI-powered financial document extraction, validation and persistence

Case Study Solution Presentation

# 1. Problem & Objective

- Build a deployed document intelligence application for Invoice, Balance Sheet, Profit & Loss and Cash Flow documents.
- Accept PDF, JPG and PNG inputs; validate type/readability and enforce a 3-page PDF limit.
- Extract meaningful visible information, including headers, dates, parties, currencies, totals, tables and comparative-period values.
- Represent missing information as null and avoid hallucinating values.
- Persist results and expose them through REST APIs and a simple dashboard.

## 2. Solution Architecture

- Frontend: HTML/CSS/JavaScript dashboard for upload, processing and result display.
- Backend: FastAPI REST API orchestrates validation, extraction, financial validation and persistence.
- Document processing: PyMuPDF and Pillow handle PDFs/images; Tesseract OCR supports scanned or image content.
- AI extraction: Google Gemini produces structured, document-specific financial data.
- Validation: Python arithmetic rules verify financial relationships with configured tolerance.
- Persistence: SQLAlchemy with SQLite stores extraction, validation and OCR results.
- Deployment: Render hosts the live application.

### 3. Document Processing Flow

- 1) User uploads a supported document.
- 2) File type, readability and page-count constraints are checked.
- 3) Native PDF text is used when available; OCR is used for scanned/image content.
- 4) Document-specific extraction logic sends content to Gemini for structured extraction.
- 5) Numeric values are cleaned and normalized.
- 6) Financial validation checks calculated relationships against reported values.
- 7) Extraction and validation results are stored in SQLite.
- 8) API and dashboard present the result, validation status and OCR/evidence information.

## 4. Extraction Coverage

- Invoice: invoice number, dates, parties, currency, line items, quantity, unit price, tax, discount, subtotal, total and payment/change where present.
- Balance Sheet: company/date, currency/unit, assets, liabilities, equity, comparative periods, totals and line items.
- Profit & Loss: income, expenses, comparative periods, profit before minority interest, minority interest, attributable group profit and other reported fields.
- Cash Flow: operating, investing and financing cash flows, FX effects, opening/closing cash and comparative periods.

## 5. Financial Validation

- Invoice: quantity  $\times$  unit price approximately equals line total; subtotal + tax – discount approximately equals total.
- Balance Sheet: assets approximately equal liabilities + equity; line-item sums and totals are checked.
- Profit & Loss: income components and expenses reconcile to reported totals; profit and minority-interest relationships are checked.
- Cash Flow: operating + investing + financing + FX approximately equals net cash change; opening and closing balances reconcile.
- Configured tolerance: absolute 0.05 and relative 0.5%; checks are evaluated independently per relevant period.

## 6. REST API

- POST /api/v1/documents/process — upload and process a document.
- GET /api/v1/documents — list persisted documents.
- GET /api/v1/documents/{document\_name} — retrieve the latest result for a document name.
- GET /api/v1/health — service health check.
- GET /docs — Swagger/OpenAPI documentation.

## 7. Dashboard & Persistence

- Simple responsive dashboard keeps the frontend lightweight and easy to deploy.
- Users select a document type, upload a file and process it.
- Results display extracted fields, line items, validation checks and OCR/evidence information.
- Processed documents can be listed and retrieved by document name.
- SQLAlchemy provides the database layer; SQLite is used for the deployed prototype.

## 8. Testing

- Automated tests cover successful processing paths, scanned/image handling, invalid file validation, financial validation and API/retrieval behavior.
- Current local automated test result: 15 passed.
- Test coverage is designed to include both successful documents and validation/error paths.

## 9. Deployment & Security

- Backend and dashboard are deployed on Render.
- The service binds to 0.0.0.0 and uses the platform-provided PORT.
- Gemini credentials are stored as environment variables and are not committed to GitHub.
- .env files, database files, virtual environments and the local dataset are excluded from the repository.
- Swagger is available from the deployed FastAPI service.

## 10. Limitations & Production Improvements

- SQLite on a free web-service deployment is suitable for a prototype; managed PostgreSQL is recommended for durable production persistence.
- OCR quality varies with scan quality, layout and resolution.
- Production should add stronger table parsing, confidence thresholds and human review for high-risk financial workflows.
- Further hardening should include authentication, rate limiting, observability, managed secrets, background processing and file scanning.
- Extraction prompts and validation rules should be versioned and regression-tested against a larger labeled dataset.

# 11. AI & Tool Usage Declaration

- AI assistants were used during development for coding assistance, debugging, architecture discussion, documentation and presentation preparation.
- The candidate should be able to explain the implemented architecture, extraction prompts, OCR approach, validation logic, API design and deployment configuration.
- Project goal: convert financial documents into structured, validated and retrievable information.